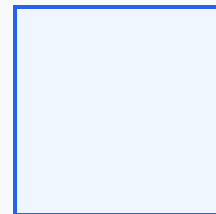


项目演进路线

阶段一：LLM 统一模型调用服务



本节会同时讲清两件事：llm-gateway 当前已经交付了什么，以及一套更完整的 LLM Gateway 在后续接入 Agent Harness 时还需要补哪些能力。

一、学习目标

完成本节后，学员应当能够：

- 1 复盘 1.6 原型的边界，说明为什么模型别名、重试、fallback、流式输出和调用审计还需要进一步工程化。
- 2 使用 FastAPI 和 Pydantic 构建 OpenAI Compatible API，让已有 OpenAI SDK 客户端可以接入网关，而上游供应商密钥留在网关内。
- 3 用 YAML 配置多个 OpenAI-compatible Provider、模型别名、优先级或加权路由，并理解重试、fallback 与熔断的执行顺序。
- 4 在同一服务中支持普通输出、SSE Streaming 和 Structured Output；理解为什么首个内容块发送后不能透明 fallback，以及为什么结构化输出仍需本地 Schema 校验。
- 5 将 Prompt 模板、变量和版本从 Python 常量中移出，使用受限模板渲染，并在调用记录中保留实际使用的 Prompt 版本。
- 6 持久化 Token、Cost、TTFT、Latency、重试、fallback 与错误信息，并建立鉴权、基础限流、健康检查和管理接口。
- 7 跑通 llm-gateway 的测试、静态检查和容器化启动，并明确它与后续 Agent Loop、Harness 的责任边界。

二、从原型到可部署服务

gateway.py 已经把消息协议、Provider Adapter、结构化输出、主备切换和 SSE 放进了一条调用链。也把供应商、密钥和响应结构带进了业务代码。



三、定义 Gateway 的职责边界与内部协议

OpenAI Compatible 是外部接口，不是内部领域模型

外部兼容接口应在 API Layer 终止。Gateway 内部使用自己的领域模型，避免业务逻辑跟随任一供应商变化。

定义请求协议

上层声明需要哪一类服务，不直接绑定 provider-x/model-2026-08。实际端点由 Router 和部署配置决定。

定义统一结果

requested_model 与 resolved_model 必须同时保留。

定义 Model Adapter 协议

错误也必须是协议。

四、构建 OpenAI Compatible API

01

API Schema

定义与 OpenAI Chat Completions 兼容的请求/响应 Schema，确保已有 SDK 无缝接入。

02

请求规范化

将外部兼容请求转换为内部领域模型，在 API Layer 边界完成协议分离。

03

FastAPI 路由

用 FastAPI 声明路由，Pydantic 模型做参数校验，供应商密钥不暴露给调用方。

04

返回响应

将内部领域模型的结果翻译回 OpenAI Compatible 响应格式，保持 requested_model 与 resolved_model。

五、多模型 Adapter、能力注册表与模型路由

能力注册表

声明每个 Adapter 支持的能力（streaming、structured output、tool calling），Router 按能力匹配而非按名称。

路由不是"挑最强模型"

路由基于能力声明、优先级、成本和健康状态，不是简单地选参数量最大的模型。

Fallback 的边界

首 Token 发送前可以 fallback；发送后发生错误则结束流，不能换模型拼接。

路由与 Eval 的关系

路由决策应可被 Eval 回放和解释，候选模型的拒绝原因必须持久化记录。

六、统一普通输出、Streaming 与 Structured Output

■ 普通调用执行器

同步请求 → 上游 → 完整响应 → 翻译回统一结果。最简单的链路，也是 fallback 和重试策略的基准。

■ Streaming 的内部事件

Gateway 内部维护事件序列 (delta / tool_call / usage / done)，对外统一为 SSE 协议。

■ 流式响应必须只有一个终态

流结束只有一个原因：正常完成、错误或取消。不能在流中间切换终态类型。

■ Structured Output 的两层保证

第一层：请求时带 JSON Schema；第二层：返回后用 Pydantic 本地校验。供应商的 Schema 保证不等于应用层安全。

■ Structured Streaming 的边界

流式 JSON 需要增量解析，不能等全部 chunk 到齐再解析。但部分 JSON 无法做 Schema 校验，需在流结束后做最终校验。

■ 输出修复要有边界

JSON 提取和修复只做有限次数的尝试，不能无限重试。修复失败应返回明确错误，不是静默吞掉。

七、Prompt 模板管理不是字符串仓库

Prompt Bundle 的组成

一个可发布 Prompt 版本至少包含：

- System Prompt 模板
- 输入变量 Schema
- Few-shot 示例
- 输出 JSON Schema
- 工具定义版本
- 默认逻辑模型和生成参数
- Context 预算
- Eval 集与通过门槛
- 变更说明和状态

使用严格模板渲染

受限模板渲染

不使用 eval 或 exec，不支持任意 Python 表达式。只允许变量替换和条件分支，模板引擎在沙箱中运行。

版本可追溯

每次调用记录中保留实际使用的 Prompt 名称、版本号和 hash，确保行为可回放。

变量校验

渲染前用 Schema 校验输入变量，缺失或超长变量直接拒绝，不进入上游请求。

九、错误处理、重试、限流与资源边界

每个阶段都有不同错误：

阶段	典型错误	是否适合重试
认证	API Key 无效、租户禁用	否
请求校验	参数非法、Schema 缺失	否
Prompt	缺变量、超过预算	修正请求后再试
路由	无兼容模型	配置变化后再试
排队	并发已满、截止时间不足	可拒绝或换池
连接	网络抖动、短暂 5xx	有限重试
首 Token 前	429、超时、过载	有限重试或 Fallback
已流式输出	连接中断	不盲目重新生成
输出校验	JSON/Schema 错误	有限修复
内容拒答	安全策略拒绝	不换模型绕过

有边界的指数退避 重试有上限，退避有 jitter	Streaming 重试边界 首 delta 前可重试，之后不可	并发 > QPS 并发限制比 QPS 更重要	幂等性边界 不能避免模型重复计费
---	--	---	--

八、Token、Cost、Latency 与 Trace

一次模型调用至少记录什么

维度	关键字段
关联	trace_id、run_id、step_id、call_id
路由	逻辑模型、供应商、实际模型、候选、拒绝原因
Prompt	名称、版本、hash、Schema 版本
用量	input、output、cached、reasoning tokens
延迟	queue、route、TTFT、generation、total latency
弹性	attempt、retry、fallback、timeout budget
结果	finish reason、终态、输出校验结果
错误	稳定错误码、HTTP 状态、供应商请求 ID
成本	输入、输出、缓存及总估算成本

TTFT 与总耗时
TTFT 必须以第一个有业务意义的 delta 为准，不能把连接建立事件当作首 Token。

指标与日志的分工
Metrics 聚合趋势（QPS、P99、错误率），Logs 记录单次调用细节，Trace 串联调用链路。

十、把 LLM Gateway 接入最小 Agent Harness

Loop 只依赖模型端口

Agent Loop 通过 Gateway 的统一接口调用模型，不直接接触供应商。Loop 负责工具调用、循环控制和状态管理，Gateway 负责模型执行、路由和弹性。

一次 Loop Step 的 Trace 关系

每个 Loop Step 生成一个 `step_id`，其下可能有多次模型调用（`call_id`）。Gateway 记录每次调用的 `trace_id`、路由决策、Token 用量和成本。Loop 层面的 `trace` 和 Gateway 层面的 `trace` 通过 `run_id` 和 `step_id` 关联，形成完整调用链。

十一、测试策略：不要只验证接口返回 200

Adapter Contract Test

验证每个 Adapter 的请求翻译和响应翻译是否符合协议，用 MockTransport 驱动，不依赖真实供应商。

重试测试不能真的等待

用 MockTransport 模拟超时和错误，验证重试次数和退避策略，但不能真的 sleep。

Structured Output 测试

验证 JSON 提取、Schema 校验、修复尝试次数、校验失败时的错误返回。

Router 测试

验证路由策略：能力匹配、优先级、fallback 顺序、候选拒绝原因是否正确记录。

Streaming 测试

验证 SSE 事件序列的正确性：delta 顺序、usage 事件、终态、错误中断行为。

可观测性测试

验证 Trace、Metrics、Cost Ledger 是否正确记录了每次调用的关键字段。

十二、项目实战：Mini LLM Gateway

本节项目结构

```
llm-gateway/
├── app/
│   ├── api/routes.py
│   │   # Chat、Responses、Prompt
│   │   # 和管理接口
│   ├── core/
│   │   # 统一错误、鉴权、限流
│   ├── services/
│   │   ├── gateway.py
│   │   │   # 调用编排、重试、流式
│   │   │   # 与结构化修复
│   │   ├── upstream.py
│   │   │   # OpenAI-compatible 客户端
│   │   ├── router.py
│   │   │   # 路由、fallback 与熔断
│   │   ├── prompts.py
│   │   │   # Prompt 版本与安全渲染
│   │   ├── structured.py
│   │   │   # JSON 提取和 Schema 校验
│   │   └── usage.py
│   │   # SQLite 用量与检查点
│   ├── config.py
│   │   # YAML 与环境变量配置
│   ├── schemas.py
│   │   # API Pydantic 模型
│   └── main.py
│   # 生命周期与依赖组装
├── tests/
│   # MockTransport 驱动测试
├── gateway.example.yaml
├── pyproject.toml
├── Dockerfile
└── docker-compose.yml
```

扩展设计

- 1 跑通内部协议与 Fake Adapter
- 2 接入真实 Adapter
- 3 Streaming 与取消
- 4 Structured Output
- 5 模型路由与 Fallback
- 6 Prompt Bundle
- 7 资源治理与观测闭环

验收标准

测试全绿、静态检查通过、Docker 可启动、API 可调用

常见面试题

Q1 为什么 OpenAI Compatible API 不能直接作为 Gateway 的内部领域模型？	Q2 LLM Gateway 与普通反向代理的本质区别是什么？
Q3 为什么逻辑模型名比直接暴露供应商模型名更适合作为平台接口？	Q4 Model Adapter、Router 与 Agent Loop 的责任分别是什么？
Q5 为什么应该关闭 SDK 内部重试并由 Gateway 统一管理？	Q6 哪些错误适合重试，哪些错误不应重试？
Q7 为什么流式输出发送首个 delta 后不能透明切换模型？	Q8 Structured Output 为什么仍需要本地 Pydantic 校验？
Q9 JSON Schema 合法为什么不等于工具调用安全？	Q10 TTFT、生成耗时和总延迟分别反映什么问题？
Q11 为什么只限制 QPS 不能保护 LLM Gateway？	Q12 如何避免重试次数在 SDK、Gateway 和 Agent Loop 三层相乘？
Q13 一次模型路由需要记录哪些信息才能解释和回放？	Q14 Prompt 版本为什么还应包含 Schema、Few-shot、工具定义和模型配置？
Q15 Gateway 如何支持灰度，同时保证同一个 Agent Run 行为稳定？	Q16 Idempotency-Key 能解决什么，为什么不一定能避免供应商重复计费？
Q17 Metrics、Logs、Traces 与 Cost Ledger 应怎样分工？	Q18 Gateway 为什么不能根据模型的 final 文本直接把任务标记为完成？

十三、本节小结

回看 1.6 和 1.7，可以把这次演进理解为：先在单文件里验证模型网关的核心语义，再把这些语义迁移到一个可配置、可部署、可治理的服务中。

- 1.6 的价值是先跑通模型白名单、Prompt 注入、Structured Output、本地校验、重试、fallback、SSE 和成本审计；
- 1.7 保留这些核心语义，但将自定义 /v1/llm 升级为 OpenAI Compatible API，使现有 SDK 和 Agent 可以直接接入；
- Provider、模型别名、路由顺序、重试参数和价格从 Python 常量移到 YAML，服务行为可以在不改业务 Agent 的前提下调整；
- 调用链被拆分为 API、鉴权限流、Prompt、路由、上游请求、结构化校验和用量存储等模块，职责比单文件原型更清楚；
- 普通调用、SSE 和 Structured Output 仍走同一条网关编排链路。首段文本发出前可以重试或 fallback；发出后发生错误则结束流，不能换模型拼接；
- Prompt、Token、Cost、TTFT、Latency、重试、fallback 和错误写入 SQLite。调用方身份只保留短指纹，默认不保存 Prompt 与消息正文；
- 当前项目已提供 API Key、进程内限流、熔断、健康检查、Docker 与自动化测试；多副本场景仍需引入共享限流和熔断状态；
- Gateway 返回的是模型执行结果。后续 Agent Harness 才负责 Run 状态、工具权限、业务校验和任务终态。